COMET: Comic Multimodal Extraction and Translation

Jay Parmar
ja339952@ucf.edu
University of Central Florida
Orlando, Florida, USA

Abstract

Manga translation is difficult because the system must handle both text and layout: it must find the text in the image, translate it, and place the new text back into the page without hurting readability. Hence, we introduce COMET, a pipeline for automatic manga translation and English re-typesetting. It solves the manga translation issue with image preprocessing, PaddleOCR-based text extraction, Japanese-to-English translation using multiple HuggingFace models, best-candidate selection for each text region, and box-constrained rendering. We evaluate the pipeline on 50 paired pages from One Piece Chapter 1. English counterpart pages are processed separately to build OCR JSON references for evaluation. The final system uses adaptive preprocessing, PP-OCRv5 for Japanese and English OCR, three translation models, and heuristic text rendering, achieving a mean box-level IoU of 0.126. This makes COMET a practical baseline for end-to-end manga translation and a useful framework for analyzing failures in OCR, translation, and re-typesetting.

Keywords

multimodal NLP, manga translation, OCR, machine translation, document layout, typesetting

Code and Artifacts: https://github.com/j-o7/manga_translator.git

1 Introduction & Problem Statement

Manual manga translation is a labor-intensive process that typically requires text extraction, translation, cleaning the original artwork, and careful re-typesetting of translated dialogue. Standard machine translation tools can translate text strings, but they do not preserve the visual structure that makes comics readable. In manga pages, dialogue is not always arranged in the same way. Text is usually inside speech bubbles, while sound effects can have unusual shapes and styles. Japanese text also makes the task harder because it is often written vertically, packed closely together, and may include furigana, which are small reading aids shown next to words. As a result, successful automatic manga translation requires joint reasoning over both text and image structure.

This project studies whether an end-to-end pipeline built from existing OCR and translation systems, combined with lightweight layout heuristics, can produce readable translated manga pages without manual intervention. The target task is not only to translate the source text, but also to place the translated content back into the page in a way that remains visually coherent.

The core problem addressed in this report is:

> How well can a modular multimodal pipeline extract Japanese text from manga pages, translate it into English, and reinsert it into the page while preserving alignment, readability, and page structure?

COMET is designed as a practical baseline rather than a trained end-to-end model. The system saves outputs from each stage, including preprocessing, OCR, translation, rendering, and evaluation. This makes the pipeline easier to understand and debug, because we can see exactly where errors happen and how they affect later stages.

The main contributions of this project are:

- A complete page-level pipeline for manga translation that combines preprocessing, OCR, translation, candidate selection, and layout-aware rendering.
- Custom OCR post-processing for Japanese and English pages, including rotation, box merging, and filtering of unwanted texts and boxes.
- A simple evaluation framework that measures layout accuracy, text accuracy, and overall end-to-end performance using aligned OCR JSON outputs.
- An analysis of 50 paired pages from *One Piece* Chapter 1, showing both successful outputs and the main failure cases in the pipeline.

2 Related Work

OCR systems are a natural starting point for manga translation because they provide both text recognition and spatial localization. PaddleOCR has shown strong multilingual OCR performance and offers practical detection and recognition models suitable for document and scene-text tasks [1]. However, manga introduces more severe challenges than standard document OCR. Stylized fonts, vertical Japanese text, irregular spacing, and visually cluttered backgrounds can all degrade detection and recognition quality.

A second line of related work comes from visually rich document understanding. Models such as LayoutLM, LayoutLMv2, and LayoutLMv3 explicitly integrate text and layout information for document reasoning tasks [3, 12, 13]. These models motivate the broader idea that document understanding should combine visual structure with linguistic content. However, their primary targets are tasks such as classification, information extraction, and question answering rather than translation and re-typesetting. Similarly, datasets such as DocVQA focus on reasoning over document images rather than reconstructing translated page layouts [6].

Manga and comic analysis has its own task-specific challenges. Prior work has studied comic speech text recognition and manga retrieval, showing that comic pages require specialized handling beyond standard document pipelines [7, 9]. In particular, speech text can be fragmented across multiple detections, text may not follow left-to-right order, and layout constraints are central to readability.

On the translation side, pretrained multilingual Transformer models have made machine translation more accessible by providing strong out-of-the-box performance across many language pairs without requiring task-specific training from scratch [2, 5]. Rather

than relying on a single model, COMET evaluates multiple off-the-shelf translation models and performs heuristic per-box selection. This design reflects the observation that translation quality can vary noticeably across short dialogue fragments, character names, and OCR-noisy text.

The key difference between COMET and the above work is that COMET treats manga translation as a full multimodal pipeline problem. The goal is not only to detect or understand page content, but to produce a translated image that preserves visual usability. This requires combining OCR, machine translation, and layout reconstruction into a single workflow. Unlike prior work that focuses on only one part of the problem, COMET also evaluates all of these stages together in a single end-to-end pipeline.

3 Methodology

3.1 Dataset

The evaluation set consists of 50 paired page images from *One Piece* Chapter 1. Japanese pages are used as source inputs to the translation pipeline, while English counterpart pages are processed separately to produce ground-truth OCR JSON files for evaluation. The pages were collected by taking screenshots of page images from online sources and organizing them into paired Japanese and English directories.

This setup made evaluation easier because it did not require us to manually draw text regions for every page. However, it also creates an important limitation. The English OCR output is not perfect ground truth. It is only an automatically generated reference based on OCR. Because of this, the final scores do not measure translation quality alone. They are also affected by mistakes in Japanese OCR, mistakes in English OCR, differences in how text regions align across languages, and errors from rendering the translated text back into the page.

3.2 Pipeline Overview

COMET follows a five-stage pipeline:

(1) First, the Japanese and English page images are preprocessed using *preprocess.py* to improve image quality for later stages.
(2) Next, OCR is run separately on the Japanese and English pages using *pp_ocrv5_jap.py* and *pp_ocrv5_en.py*. This stage also refines and merges the detected text boxes.
(3) The extracted Japanese text is then translated into English using multiple JA→EN models through *translate_jp_to_en.py*.
(4) After that, the system chooses the best translation for each text region and renders it back into the page using *typeset.py*.
(5) Finally, the predicted JSON outputs are compared with the English reference OCR JSON files using *eval.py*.

Figure 1 shows the main stages of the final COMET pipeline.

3.3 Image Preprocessing

Preprocessing standardizes the page images before OCR. In *preprocess.py*, each image is first loaded safely, resized while keeping its aspect ratio, and denoised with `fastNlMeansDenoising`. The script then applies one of several preprocessing techniques: grayscale, CLAHE, Otsu thresholding, inverted Otsu, adaptive thresholding, inverted adaptive thresholding, background normalization, or a mild binary variant with morphological closing. It also saves a resized original copy of each page, which is later reused during rendering.

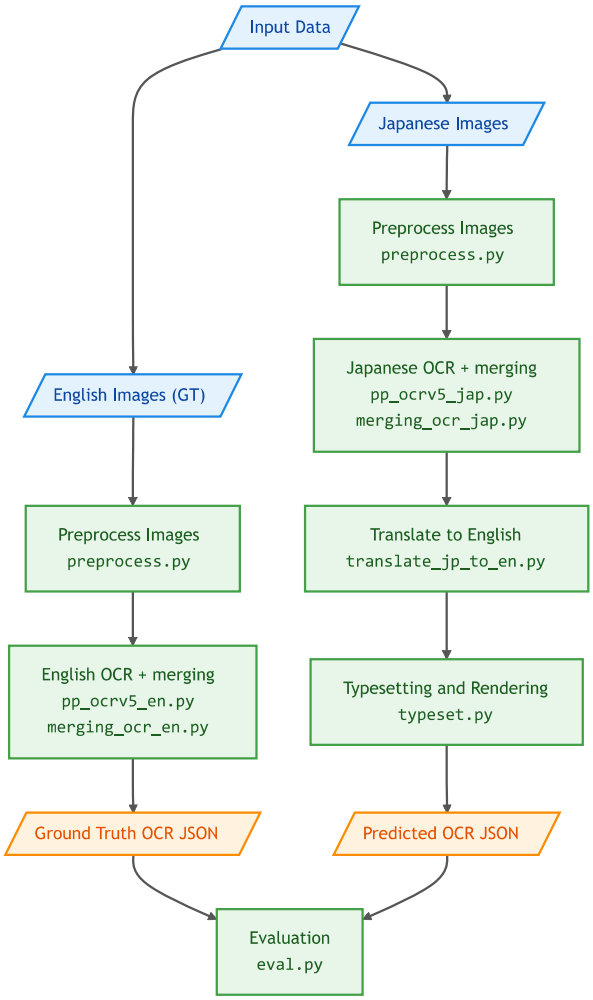

Figure 1: COMET pipeline for manga translation and evaluation. Japanese pages are preprocessed, passed through OCR, translated into English, and re-typeset to produce predicted OCR JSON. English counterpart pages are processed separately to create reference OCR JSON, and both outputs are compared during evaluation.

The goal of preprocessing is to improve text visibility before OCR while keeping the page structure stable across later stages. In our experiments, adaptive thresholding gave the best overall OCR performance, so it was used in the final pipeline for both Japanese and English pages. However, other methods were still useful in some cases, especially for pages with heavily stylized or unusual text. A full grid search over all preprocessing methods for the complete end-to-end pipeline has not yet been carried out, so the current choice should be viewed as the best practical setting found so far rather than a fully optimized one. This is also consistent with the current pipeline setup, where adaptive preprocessing is used as the default run configuration.

Figure 2: Qualitative example from COMET. The sequence illustrates page processing across intermediate stages, including source page, OCR localization, and final translated rendering. The example shows that the system can produce readable English output while approximately preserving panel structure and dialogue placement.

3.4 OCR and Box Refinement

OCR is one of the most important parts of COMET because all later stages depend on it. The pipeline runs PP-OCRv5 [1] separately on the Japanese source pages and the English reference pages. These two branches use the same OCR model, but they have different goals. The Japanese branch prepares text regions for translation, while the English branch creates cleaner OCR JSON files for evaluation.

For Japanese pages, raw OCR output is often broken into many small boxes, especially when the text is vertical, tightly packed, or mixed with small reading aids such as furigana. To make the OCR output more useful for translation, COMET passes these detections through a heuristic merging pipeline that:

(1) Removes boxes that are clearly not useful, such as empty text, very small regions, symbol-only detections, and some isolated single-character boxes.
(2) Uses the shape of each bounding box to separate likely vertical text from likely horizontal text.
(3) Groups and merges nearby boxes using simple layout cues such as overlap, distance between boxes, and center alignment.
(4) Removes duplicate detections when two boxes contain nearly the same text and strongly overlap.
(5) Attaches small nearby text that looks like furigana to the main text block instead of treating it as separate dialogue.

Figure 2 shows this process on a sample page. The intermediate stages highlight how the raw OCR regions are cleaned and merged before translation. After merging, the final Japanese texts are sorted into reading order, i.e. right to left, and saved as translation-ready JSON files. This produces text regions that are closer to complete dialogue units and are therefore better suited for box-level translation than the original raw OCR output.

For English pages, *pp_ocrv5_en.py* first restores the OCR boxes to the original page orientation, since the OCR preprocessor may rotate the image internally. It then merges nearby English boxes using center and gap rules and removes boxes that are too small, single-character, or not English-like. The result is a cleaner page-level JSON file that can be used as a reference during evaluation.

Both branches save intermediate images and JSON outputs. This makes the OCR stage easier to debug and allows direct inspection of raw detections, merged regions, and final OCR text before translation and evaluation.

3.5 Translation with Multiple JA→EN Models

The translation stage takes the refined Japanese OCR JSON and runs three Japanese-to-English models in parallel:

- facebook/m2m100_418M [2]
- Helsinki-NLP/opus-mt-ja-en [11]
- facebook/nllb-200-distilled-600M [10]

In the code, each OCR region is translated separately, and each model writes a JSON output with the same page structure as the original OCR input. The script also normalizes text, keeps the source Japanese strings for debugging, and removes some repetitive translation artifacts before saving the results.

These models were chosen to provide different strengths within the limits of available GPU memory. M2M100 [2] is a many-to-many multilingual translation model that directly supports translation across 100 languages, including Japanese and English. OPUS-MT [11] is a direct Japanese-to-English Marian model and serves as a bilingual baseline. NLLB-200 [10] is a large multilingual translation family designed for broad language coverage and stronger translation quality across many language pairs; in this project, the

distilled 600M variant was used as a smaller practical option. Together, these models provide a useful mix of multilingual coverage, direct JA→EN translation, and manageable runtime cost.

This multi-model design is helpful because manga OCR text is often short, noisy, and inconsistent. Some boxes contain full dialogue, while others contain short reactions, fragmented text, or OCR errors. Instead of relying on one model for every case, COMET generates multiple candidate translations and lets the later typesetting stage choose the best candidate for each region. In the current implementation,the translated JSON files also carry per-region scores that are used in later candidate selection and rendering steps. Ideally, the confidence scores should be a separate evaluation pipeline against the ground truth.

3.6 Typesetting and Rendering

After translation, COMET places English text back into the original manga page. The rendering is box-based rather than bubble-segmented, meaning that each translated text region is assigned a writable area based on the OCR polygon or bounding box instead of explicitly detecting the full speech bubble shape. This makes the method simpler and easier to apply to many pages, although it can be less precise when the bubble shape is irregular.

The *typeset.py* script first compares the translations produced by the three models for each text region. It chooses a candidate only if the text appears to be English, using simple checks such as the ratio of English letters and whether Japanese characters remain. Among the valid English candidates, the script selects the one with the highest score carried in the translated JSON and stores the chosen text along with its model name for debugging. The selected translations are then rendered back into the page.

The rendering stage includes the following steps:

(1) selecting the best translation candidate for each region using English-likeness and per-region scores,
(2) converting the OCR polygon into a rectangular writable area,
(3) wrapping the translated text to fit the box width,
(4) iteratively reducing the font size until the text fits within the assigned region,
(5) centering the text inside the box,
(6) whitening only the text footprint with a small amount of padding rather than repainting the whole box, and
(7) merging overlapping render boxes when necessary.

In the final run, the script used a minimum font size of 10, a maximum font size of 42, and a padding of 1 pixel. When regions overlap, the corresponding texts are merged and rendered together in a larger box. For debugging, the script also records which model was selected for each region and whether a region was merged with another. Figure 2 shows an example page across these stages, including the OCR detections, merged text regions, and the final English rendering.

One important limitation is that PaddleOCR [1] performs its own internal document preprocessing, such as rotation and removal of surrounding white space. This changes the coordinate system used by OCR. As a result, the detected boxes can be slightly misaligned when they are drawn back onto the original page image. This effect can be seen in Figure 2, where some re-rendered text appears slightly shifted from the ideal speech region. A possible improvement would be to render directly on PaddleOCR's preprocessed output image, which would better match the OCR coordinates. However, that image has already been altered by both PaddleOCR and our own preprocessing methods (such as adaptive or Otsu thresholding), whereas our goal is to place translated text back into the original page itself. This creates a trade-off between cleaner alignment and preserving the original artwork. In principle, this trade-off is fixable through a deeper analysis of the PaddleOCR code, preprocessing steps and output, which could make it possible to map the OCR coordinates back more accurately onto the original image.

This stage produces both rendered page images and predicted OCR-style JSON outputs, allowing the same pipeline to support qualitative inspection and quantitative evaluation.

3.7 Evaluation Protocol and Metrics

Evaluation compares the predicted translated JSON files with the English reference JSON files at the text-region level. For each ground-truth box, *eval.py* first selects the predicted box with the highest IoU. If the overlap is too low, it falls back to the box with the smallest center distance. This fallback is needed because Japanese and English text regions often have different shapes, especially when vertical Japanese text is rendered as horizontal English text. The final run used an IoU threshold of 0.1, and the script outputs both per-box and page-level CSV summaries.

COMET reports both geometry-based and text-based metrics. Geometry-based metrics measure whether the translated text is placed in the correct part of the page, while text-based metrics measure how close the predicted text is to the English reference. Using both types of metrics makes it easier to separate layout errors from translation errors.

Intersection over Union (IoU)..

$$\mathrm{IoU}(G, P) = \frac{|G \cap P|}{|G \cup P|} \tag{1}$$

IoU measures how much the predicted region overlaps with the ground-truth region. A high IoU means the translated text was placed in roughly the correct location.

Intersection over Ground Truth (IoGT)..

$$\mathrm{IoGT}(G, P) = \frac{|G \cap P|}{|G|} \tag{2}$$

IoGT measures how much of the ground-truth region is covered by the prediction. This is useful when the predicted box only covers part of the correct region or when the two boxes have different shapes.

Center Distance. Center distance measures the Euclidean distance between the centers of the ground-truth and predicted polygons. This metric is mainly used during fallback matching, but it is also useful for understanding how far a prediction is from the correct location when overlap is low.

Normalized Levenshtein Similarity. [4].

$$\mathrm{LevSim}(t_{gt}, t_{pred}) = 1 - \frac{\mathrm{ED}(t_{gt}, t_{pred})}{\max(|t_{gt}|, |t_{pred}|, 1)} \tag{3}$$

where ED is edit distance. This metric measures character-level text similarity after light normalization. It is useful because manga dialogue may differ by only a few characters even when the overall meaning is similar.

Character-Level Similarity and Word-Level Similarity. The evaluator also reports a CER-like similarity and a word-level similarity. These provide additional views of textual correctness. Character-level similarity is sensitive to small OCR or translation mistakes, while word-level similarity is more useful when whole words are inserted, removed, or changed.

chrF. chrF [8] is a character n-gram F-score. It gives a softer text comparison than exact edit distance and is helpful when the prediction is only partially correct.

End-to-End Weighted Text Metrics.

$$\text{E2EText} = \text{IoU}(G, P) \times \text{TextScore}(t_{gt}, t_{pred}) \tag{4}$$

COMET also reports end-to-end versions of the text metrics by multiplying text similarity with IoU. The reason for this is simple: a text prediction should not receive a high score if it is textually similar but placed in the wrong part of the page. These weighted metrics therefore reward predictions that are both correct in content and reasonably correct in location.

Recall at IoU Thresholds.

$$\text{Recall@}\tau = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{IoU}_i \geq \tau] \tag{5}$$

COMET reports Recall@0.3 and Recall@0.5. These values show the fraction of text regions that reach at least moderate or strong overlap with the reference.

Page-Level Concatenated Text Metrics. In addition to per-box scores, the evaluator concatenates all matched text on a page in ground-truth order and computes page-level Levenshtein similarity [4] and chrF [8]. These metrics are useful because even if individual boxes are imperfectly aligned, the page may still preserve much of the overall dialogue content. We also use length-weighted text metrics so that longer dialogue regions contribute more than very short ones.

4 Evaluation & Results

The final experiment used the full pipeline on 50 paired pages from *One Piece* Chapter 1. Japanese pages were taken from *data/onepiece/jap* and English pages from *data/onepiece/en*. The run used adaptive preprocessing, PP-OCRv5 [1] for both Japanese and English OCR, three translation models (M2M100 [2], OPUS-MT [11], and NLLB [10]), and an evaluation IoU threshold of 0.1. The English reference side contained 510 text regions, which is an average of 10.2 regions per page.

4.1 Quantitative Results

Table 1 summarizes the main results. These numbers should be read with care because the English side is OCR-based ground truth rather than manual annotation.

The results show three main patterns. First, layout alignment is the weakest part of the pipeline. The IoU scores are low, which means the predicted English text often does not match the reference region shape closely. This is expected because Japanese text is often vertical and compact, while the rendered English text is usually more horizontal. The mismatch is also affected by PaddleOCR's internal preprocessing [1].

Table 1: Main quantitative results on 50 paired pages from *One Piece* Chapter 1.

Category	Metric	Value
Dataset	Number of pages	50
Dataset	Total evaluated GT boxes	510
Dataset	Mean GT boxes per page	10.2
Geometry	Mean IoU	0.126
Geometry	Mean IoGT	0.192
Geometry	Mean center distance	139.256
Geometry	Recall@0.3	0.135
Geometry	Recall@0.5	0.045
Text	Mean box-level Levenshtein similarity	0.187
Text	Mean box-level chrF	0.139
Text	Mean page-level Levenshtein similarity	0.267
Text	Mean page-level chrF	0.253
Text	Best page-level chrF	0.372
Text	Lowest page-level chrF	0.039
End-to-end	Mean end-to-end Levenshtein	0.029
End-to-end	Mean end-to-end chrF	0.022

Second, text similarity is better at the page level than at the box level. Mean page-level Levenshtein similarity [4] and chrF [8] are both higher than their box-level versions, which suggests that the system often preserves some overall page content even when individual text regions are not matched well. This can also be seen in the spread of page-level chrF [8] scores, which range from 0.039 to 0.372.

Third, the end-to-end scores are much lower than the text-only scores because they also penalize poor geometric alignment. This shows that translation quality alone is not enough; the text must also be placed in the correct part of the page.

Matching behavior also supports this point Out of 510 evaluated regions, 225 were matched directly by IoU, while 285 required center-distance fallback. This shows that overlap alone was often not enough for cross-language alignment. Overall, the results varied widely across pages, reflecting the difference between easier pages with clear dialogue regions and harder pages with dense or stylized text.

4.2 Qualitative Results

Figure 2 shows a representative example from COMET, including the original page, intermediate OCR and merged text regions, and the final English rendering. The figure shows that the pipeline can preserve much of the page structure while inserting translated text into approximate dialogue regions.

Qualitatively, COMET works best on pages with clear bubbles, moderate text density, and larger, less stylized text. It struggles more on pages with dense dialogue, bold or decorative text, fragmented

vertical OCR regions, and complex backgrounds. In these harder cases, OCR errors often carry into translation and rendering, leading to short, noisy, or poorly aligned English output.

4.3 Interpretation of Results

The main takeaway from the results is that COMET is a workable proof-of-concept for end-to-end manga translation, but it is not yet a high-accuracy system. The pipeline successfully demonstrates full data flow from source image to translated rendered output, which is a meaningful systems contribution. However, the quantitative results make it clear that OCR and region alignment remain the central problems.

The gap between page-level text metrics and box-level geometric metrics is especially important. It suggests that evaluation should not rely on a single scalar score. A system may produce partially correct page content while still receiving very low end-to-end scores because cross-language region geometry differs. For this project, reporting both geometry-aware and text-aware metrics gives a more honest picture of performance.

5 Discussion & Challenges

COMET shows that end-to-end manga translation is possible with a modular pipeline, but it also makes the main challenges clear. The biggest bottleneck is OCR. Manga text is often vertical, stylized, tightly packed, or mixed with furigana, so the OCR output may contain missing boxes, duplicate detections, or fragmented text. Once this happens, the errors continue into translation and rendering, which lowers the final quality.

Another challenge is evaluation. The English side is created with OCR instead of manual annotation, so some low scores come from OCR mismatch rather than true translation failure. In addition, translation is currently done one text region at a time, which means the system does not use dialogue context from nearby boxes. The rendering stage also has its own limit: it is box-based rather than bubble-aware, so it can struggle when speech regions are curved, irregular, or visually complex.

Even with these limitations, a system like COMET is still useful. Manga and comic localization requires a large amount of manual work, especially for text extraction, translation drafting, and re-typesetting. A practical pipeline does not need to replace human editing completely to be valuable. It can still reduce time and effort by producing a strong first draft that editors or translators can refine later.

There is also a clear industry need for this kind of application. According to Grand View Research, the global comic book market was estimated at about USD 19.05 billion in 2025. In the same report, manga was listed as the largest genre segment, which shows the scale and relevance of this space. More importantly, the same pipeline is not limited to Japanese manga. If adapted carefully, it could also be used for comic books and graphic novels in multiple languages, since the main stages (OCR, translation, region grouping, and re-typesetting) are useful beyond a single language pair.

There are several clear directions for improvement. On the OCR side, better preprocessing, stronger vertical-text grouping, improved furigana handling, and better merging heuristics could reduce fragmentation. A deeper study of PaddleOCR's internal preprocessing and coordinate transformations could also improve alignment when rendering back onto the original page. On the translation side, the system could be improved by fine-tuning on manga-style Japanese-English data, adding context across nearby dialogue boxes, and using better reranking methods to choose among candidate translations. More advanced models with cross-attention between image regions and text could also help the system use both layout and dialogue content together. On the rendering side, bubble-aware segmentation, better masking or inpainting, and smarter text fitting could improve readability. Finally, evaluation could be improved with a manually verified subset, controlled preprocessing ablations, and stronger end-to-end comparisons. Together, these changes would make the system more accurate, easier to evaluate, and more useful in real translation workflows.

6 Concluding Remarks

We presented COMET, a modular pipeline for automatic manga translation and layout-aware English re-typesetting. The system combines adaptive preprocessing, PP-OCRv5-based OCR, multiple Japanese-to-English translation models, heuristic candidate selection, constrained rendering, and a geometry-plus-text evaluation framework. Together, these stages form a complete end-to-end workflow that can take a Japanese manga page, extract its text, translate it, and place the translated text back into the page in a readable form. One of COMET's main strengths is that it makes each stage visible and debuggable, which helps identify where errors begin and how they affect the rest of the pipeline.

The results on 50 paired pages from *One Piece* Chapter 1 show that COMET is a useful baseline, but the task is still difficult. The system produced readable translated pages in many cases, and the page-level text scores were better than the strict box-level overlap scores. This means COMET can often preserve the overall page content even when individual text regions are not matched perfectly.

At the same time, the main limitations are clear. OCR quality and region alignment are still the biggest problems. Japanese manga text is often vertical, dense, and stylized, which makes OCR more likely to miss or split text. The English reference side is also OCR-based rather than manually annotated, so some low scores come from OCR mismatch and layout differences, not only from translation errors.

Overall, the project shows that end-to-end manga translation is feasible with a modular pipeline. With better OCR, translation, and layout modeling, the same approach could also be adapted to comic books and graphic novels in multiple languages. In this sense, COMET is valuable both as a working system and as a foundation for future improvements in multimodal document translation.

7 AI Usage

In the early stages, AI was used to explore possible OCR models, translation models, and general project steps after providing the task context. During coding, they were mostly helpful for debugging, checking small code changes, and generating alternate versions of helper functions, especially for the merging and typesetting pipelines. AI was also used to help search for research papers on evaluation metrics.

References

[1] Cheng Cui, Ting Sun, Manhui Lin, Tingquan Gao, Yubo Zhang, Jiaxuan Liu, Xueqing Wang, Zelun Zhang, Changda Zhou, Hongen Liu, Yue Zhang, Wenyu Lv, Kui Huang, Yichao Zhang, Jing Zhang, Jun Zhang, Yi Liu, Dianhai Yu, and Yanjun Ma. 2025. PaddleOCR 3.0 Technical Report. arXiv:2507.05595 [cs.CV] https://arxiv.org/abs/2507.05595

[2] Angela Fan, Shruti Bhosale, Holger Schwenk, Zhiyi Ma, Ahmed El-Kishky, Siddharth Goyal, Mandeep Baines, Onur Celebi, Guillaume Wenzek, Vishrav Chaudhary, Naman Goyal, Tom Birch, Vitaliy Liptchinsky, Sergey Edunov, Edouard Grave, Michael Auli, and Armand Joulin. 2020. Beyond English-Centric Multilingual Machine Translation. arXiv:2010.11125 [cs.CL] https://arxiv.org/abs/2010.11125

[3] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. arXiv:2204.08387 [cs.CL] https://arxiv.org/abs/2204.08387

[4] Vladimir I. Levenshtein. 1965. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics. Doklady* 10 (1965), 707–710. https://api.semanticscholar.org/CorpusID:60827152

[5] Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual Denoising Pre-training for Neural Machine Translation. arXiv:2001.08210 [cs.CL] https://arxiv.org/abs/2001.08210

[6] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. 2021. DocVQA: A Dataset for VQA on Document Images. arXiv:2007.00398 [cs.CV] https://arxiv.org/abs/2007.00398

[7] Yusuke Matsui, Kota Ito, Yuji Aramaki, Azuma Fujimoto, Toru Ogawa, Toshihiko Yamasaki, and Kiyoharu Aizawa. 2016. Sketch-based manga retrieval using manga109 dataset. *Multimedia Tools and Applications* 76, 20 (Nov. 2016), 21811–21838. https://doi.org/10.1007/s11042-016-4020-z

[8] Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, Ondřej Bojar, Rajan Chatterjee, Christian Federmann, Barry Haddow, Chris Hokamp, Matthias Huck, Varvara Logacheva, and Pavel Pecina (Eds.). Association for Computational Linguistics, Lisbon, Portugal, 392–395. https://doi.org/10.18653/v1/W15-3049

[9] Christophe Rigaud, Srikanta Pal, Jean-Christophe Burie, and Jean-Marc Ogier. 2016. Toward speech text recognition for comic books. In *Proceedings of the 1st International Workshop on CoMics ANalysis, Processing and Understanding* (Cancun, Mexico) *(MANPU '16)*. Association for Computing Machinery, New York, NY, USA, Article 8, 6 pages. https://doi.org/10.1145/3011549.3011557

[10] NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. 2022. No Language Left Behind: Scaling Human-Centered Machine Translation. arXiv:2207.04672 [cs.CL] https://arxiv.org/abs/2207.04672

[11] Jörg Tiedemann, Mikko Aulamo, Daria Bakshandaeva, Michele Boggia, Stig-Arne Grönroos, Tommi Nieminen, Alessandro Raganato, Yves Scherrer, Raul Vazquez, and Sami Virpioja. 2023. Democratizing Neural Machine Translation with OPUS-MT. arXiv:2212.01936 [cs.CL] https://arxiv.org/abs/2212.01936

[12] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. LayoutLM: Pre-training of Text and Layout for Document Image Understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery amp; Data Mining (KDD '20)*. ACM, 1192–1200. https://doi.org/10.1145/3394486.3403172

[13] Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, Min Zhang, and Lidong Zhou. 2022. LayoutLMv2: Multi-modal Pre-training for Visually-Rich Document Understanding. arXiv:2012.14740 [cs.CL] https://arxiv.org/abs/2012.14740